16: THE JAVA COLLECTIONS FRAMEWORK

Introduction

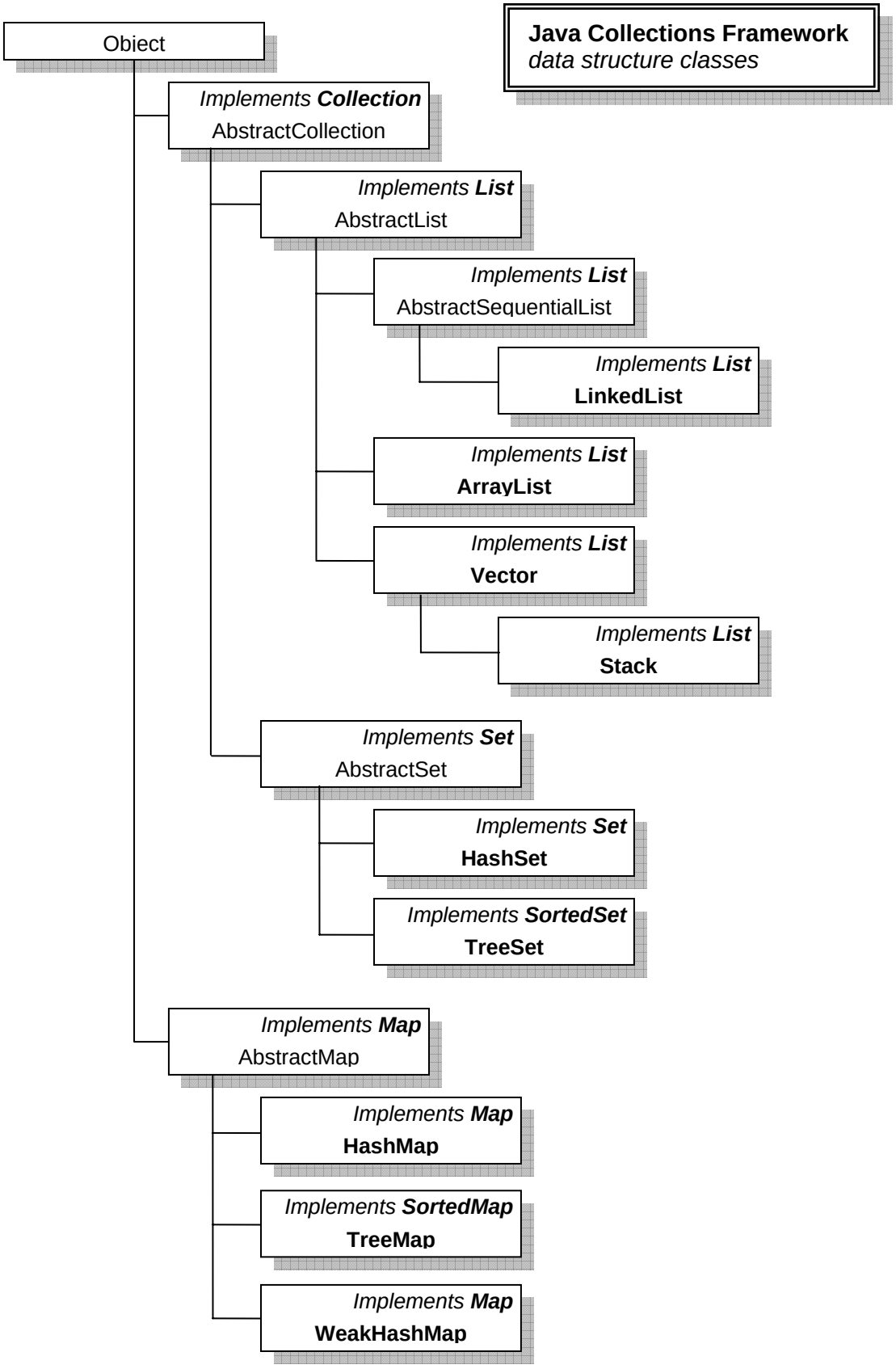
The **Java Collections Framework** is a set of classes which implement a useful range of data structures and algorithms. It is usually better to use the ready-made framework classes in applications than to implement your own data structure classes from scratch, as it will save development time and the framework classes are tried and tested. The framework was introduced in Java 2 SE v 1.2.

The Collections Framework, which is in the ***java.util*** package, defines eight interfaces for data collections. These interfaces specify the operations which each general type of collection has. The actual classes, such as *LinkedList*, implement the relevant interface. In other words, the *LinkedList* class implements all the operations which are specified in the *List* interface.

Interface	Description
Collection	A collection of elements
List (extends Collection)	A sequence of elements
Set (extends Collection)	A collection of unique elements
SortedSet (extends Set)	A sorted collection of unique elements
Map	A collection of (key, value) pairs - keys must be unique
SortedMap (extends Map)	A sorted collection of (key, value) pairs
Iterator	An object that can traverse a collection
ListIterator (extends Iterator)	An object that can traverse a sequence

List is a subinterface of *Collection*, and so on. A subinterface is an interface which extends a parent interface, just like a subclass extends a class.

The chart shows the data structure classes in the Framework, the relationships between the classes, and the interfaces implemented.


Java Collections Framework
data structure classes
Object
Implements Collection
AbstractCollection
Implements List
AbstractList
Implements List
AbstractSequentialList
Implements List
LinkedList
Implements List
ArrayList
Implements List
Vector
Implements List
Stack
Implements Set
AbstractSet
Implements Set
HashSet
Implements SortedSet
TreeSet
Implements Map
AbstractMap
Implements Map
HashMap
Implements SortedMap
TreeMap
Implements Map
WeakHashMap

For example, *AbstractList* is an abstract class which extends *AbstractCollection*, while ***ArrayList*** is a concrete class which extends *AbstractList*. You can create an instance of *ArrayList* in an application.

Using the interfaces

If an object reference is declared with type *List*, then the runtime type can be any class which extends *AbstractList*, including the framework classes *LinkedList*, *ArrayList*, *Vector* or *Stack*.

The *Collection* interface specifies the minimum set of methods which a data collection class must have:

```
public interface Collection {
    int size();
    boolean isEmpty();
    boolean contains(Object o);
    Iterator iterator();
    Object[] toArray();
    Object[] toArray(Object a[]);
    boolean add(Object o);
    boolean remove(Object o);
    boolean containsAll(Collection c);
    boolean addAll(Collection c);
    boolean removeAll(Collection c);
    boolean retainAll(Collection c);
    void clear();
    boolean equals(Object o);
    int hashCode();
}
```

Other interfaces specify additional methods for more specialised uses.

Look at the Java API documentation for more details of the Collections Framework classes and interfaces and the methods they support.

New additions to the Collections Framework

Java 2 SE v 5.0, introduced in late 2004, has added some new classes to the Collections Framework, including a Queue interface and a number of specialised queue classes, including *PriorityQueue*, *ConcurrentLinkedQueue*, *ArrayBlockingQueue* and *LinkedBlockingQueue*. In addition, the *LinkedList* class has now been modified and can be used as a *Queue*.

Map Classes

The *Map* classes store *(key, value)* pairs.

- **HashMap** stores *(key, value)* pairs in a hash table which allows very efficient searching
- **TreeMap** stores *(key, value)* pairs in a tree structure – stores the elements in order and allows binary searching

The basis of a hash table is a hash function. A hash function is a function that takes an element of whatever data type you are storing (integer, string, etc) and outputs an integer in a certain range. This integer is used to determine the location in the table for that element. The hash table itself consists of an array whose indexes are the range of the hash function.

EXERCISE: Using a Collections Framework class

The *java.util.Stack* class does a similar job to the *Stack* class you created in chapter 12, so in this exercise you will use *java.util.Stack* instead of your own in the same application.

Modify your stacks project as follows:

- Remove the *Stack* and *StackTester* classes.
- Add the following line at the top of the *BracketChecker* class:

```
import java.util.Stack;
```

- Change the declaration of the stack in *BracketChecker* to

```
private Stack myStack = new Stack();
```

Compile the class and repeat the test you did in the exercise in chapter 12

EXERCISE: Choosing a data structure

The efficiency of an application and the ease with which it can be developed can depend strongly on the data structures used.

What data structures would you choose for the following scenarios?

Your data items must be processed one at a time, and incoming data may have to wait until the previous data has been processed.

The most important feature of your application is fast retrieval of specific data items.

New data will often be added at the start of a large collection.

It is important to be able to traverse your data in order of a key value.

New data will often be added at the end of a large collection.

New data will often be added at random positions in a large collection.

Your data must be stored temporarily and the last data item stored must be retrieved first.